
□ HealthAI: Working Methods and Result Explanation

📁 Based on `app.py` (Flask Backend) & `streamlit_app.py` (Streamlit Frontend)

✓ Section 1: Overview of HealthAI Architecture

HealthAI is a full-stack healthcare assistant powered by IBM's **Granite-13b-instruct-v2** LLM. It is divided into:

Layer	Technology Used	Purpose
Frontend	Streamlit	User interface for input/output
Backend	Flask (Python)	Serves REST API endpoints to process user inputs
AI Model API	IBM Granite via HTTP POST	Handles AI-driven responses via prompts
Security	dotenv, environment vars	Secures API keys and project IDs

🔒 Section 2: Secure Setup – Environment & Credentials Handling

.env Configuration:

```
IBM_API_KEY="your_ibm_api_key"
```

```
IBM_PROJECT_ID="your_project_id"
```

In `app.py`:

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
ibm_api_key = os.getenv("IBM_API_KEY")
```

```
ibm_project_id = os.getenv("IBM_PROJECT_ID")
```

✓ Result:

- Credentials are **not hard-coded**, ensuring **security and flexibility** for local and cloud deployment.

Section 3: Flask Backend — Core Functionality

The Flask app defines multiple REST API routes that process user input and invoke the IBM Granite API to generate AI responses.

Endpoint 1: /chat

Purpose: Responds to user health-related questions using IBM Granite.

How it works:

1. Receives a POST request with JSON: {"message": "user's question"}
2. Formats the prompt with medical disclaimer.
3. Calls `get_granite_response(prompt)`.
4. Returns AI-generated response.

```
@app.route('/chat', methods=['POST'])
```

```
def chat_with_ai():
```

```
    ...
```

```
    prompt = f"As a compassionate healthcare assistant... Question: '{user_message}'"
```

```
    ai_response = get_granite_response(prompt)
```

```
    return jsonify({"response": ai_response})
```

Result:

- HealthAI provides clear, informative answers for general health questions.
- Emphasizes **non-substitution** for professional medical advice.

□ Section 4: Disease Prediction Logic

Endpoint 2: /predict_disease

Purpose: Predict potential health conditions from user-reported symptoms.

How it works:

1. Accepts JSON input: {"symptoms": "fatigue, headache", "profile": {"age": 35, "gender": "Male"}}
2. Converts patient profile into JSON string for the prompt.
3. Invokes the IBM model with a diagnostic-type prompt.
4. Returns probable conditions and next steps.

```
prompt = (  
    f"A user describes the following symptoms: '{symptoms}'. The user's profile is: {profile_str}. "  
    f"List potential conditions... and emphasize that a medical diagnosis is essential."  
)
```

✓ Result:

- Empowers users to **understand potential causes** for their symptoms.
- Acts as a **preliminary guide** to initiate further medical consultation.

Section 5: Treatment Plan Generator

Endpoint 3: /generate_treatment_plan

Purpose: Deliver AI-generated treatment strategies based on diagnosed condition and patient profile.

Workflow:

1. Accepts JSON: {"condition": "Hypertension", "profile": {...}}
2. Prompts the LLM to return:
 - Medication suggestions
 - Lifestyle changes
 - Follow-up recommendations
3. Cautions that it's AI-generated and needs professional review.

```
prompt = (  
    f"Based on the diagnosed condition '{condition}' and the patient profile: {profile_str}, "
```

```
f"create a comprehensive treatment plan..."
```

```
)
```

✓ **Result:**

- Gives a **personalized, structured plan** for common chronic diseases.
 - Enhances user understanding of condition management.
-

☑ **Section 6: Health Analytics & Insights**

🌐 **Endpoint 4: /health_analytics_insights**

Purpose: Offer feedback on user health trends based on metrics data (e.g., heart rate, BP, glucose).

Workflow:

1. Accepts health data JSON and optional profile.
2. Prompts the LLM to:
 - Detect trends
 - Flag abnormalities
 - Suggest improvements
3. Returns health insight in natural language.

prompt = (

```
f"Analyze the following health metrics: {data_str}..."
```

```
f"Provide actionable recommendations."
```

```
)
```

✓ **Result:**

- Acts as a **virtual health coach** by analyzing time-series-like data.
 - Helps users monitor and improve daily health decisions.
-

☐ **Section 7: IBM Granite Model Integration**

Function: `get_granite_response(prompt_text: str)`

Core Method: Sends prompt to IBM's Granite-13b-instruct-v2 model via HTTP POST.

How it works:

- Constructs headers with authorization, model ID, project ID.
- Payload includes:
 - Input prompt
 - Decoding method: greedy
 - Token limits
- Parses and returns the generated_text.

```
response = requests.post(API_URL, headers=headers, json=payload, timeout=60)
```

```
return response_data["results"][0].get("generated_text")
```

Result:

- Supports deterministic, reliable responses tailored to healthcare.
- Efficient integration with low-latency API calls.

End of Part 1 (Lines 1–120)

Would you like me to continue with **Part 2**, covering the Streamlit frontend (`streamlit_app.py`), error handling, UI features, user flow, and final outcomes?